

1. Deep Learning & Deep RL Architectures for Central Risk Book (CRB) Management

Sitting across from an Executive Director (ED) on Nomura's Cash Equities Central Risk Book (CRB) desk, you must articulate a precise understanding of high-frequency sequence modeling and continuous risk management under state transitions. The CRB functions as an internal aggregator of institutional flow, processing large blocks internalized from client orders. Managing this inventory requires optimizing execution scheduling and risk offloading while minimizing market impact and adverse selection.

Standard statistical frameworks assume structural stability. In contrast, modern quantitative research frameworks leverage Deep Reinforcement Learning (DRL) and Deep Learning (DL) sequence models to navigate microstructural dynamics and regime shifts.

2. Mathematical Foundations from First Principles

Receptive Field Evolution in Temporal Convolutional Networks (TCNs)

To capture multi-scale temporal dependencies in limit order book (LOB) dynamics without the memory degradation or vanishing gradients of traditional LSTMs, we deploy Temporal Convolutional Networks (TCNs) using **Dilated Causal Convolutions**.

A 1D causal filter ensures that the output at time step t depends only on inputs from times $t, t-1, \dots, 0$, eliminating future data leakage. For a 1D sequence input $\mathbf{x} \in \mathbb{R}^T$ and a filter/kernel $\mathbf{w} = [w_0, w_1, \dots, w_{k-1}]^\top \in \mathbb{R}^k$ of size k , the dilated causal convolution operation $*$ at index t under a dilation factor $d \in \mathbb{Z}^+$ is defined as:

$$(\mathbf{x} *_d \mathbf{w})(t) = \sum_{i=0}^{k-1} w_i \cdot x_{t-d \cdot i}$$

Step-by-Step Mathematical Line-by-Line Dissection of the Receptive Field (RF)

The receptive field defines the historical lookback window captured by a specific network architecture layer. Let k_l denote the kernel size at layer l , and let d_l represent the dilation factor at layer l .

Step 1: Define the base receptive field of the first layer ($l = 1$), assuming a default base dilation factor of $d = 1$:

$$RF_1 = k_1$$

Step 2: For the second layer ($l = 2$) with a dilation factor d_2 , each element in its kernel spans a distance of d_2 steps over the output elements of layer 1. The total step distance added by this layer is $(k_2 - 1) \cdot d_2$. Thus, the cumulative receptive field becomes:

$$RF_2 = RF_1 + (k_2 - 1) \cdot d_2 = k_1 + (k_2 - 1) \cdot d_2$$

Step 3: Generalize this structural induction to an arbitrary deep layer configuration L . The receptive field scales additively with each subsequent layer's expanded span:

$$RF_L = 1 + \sum_{l=1}^L (k_l - 1) \cdot d_l$$

Step 4: On the CRB desk, we parameterize the network using an exponentially growing dilation schedule: $d_l = O(2^{l-1}) = 2^{l-1}$. Assuming a uniform kernel size $k_l = k$ across all operational layers, substitute this exponential dilation schedule into the general expression:

$$RF_L = 1 + \sum_{l=1}^L (k - 1) \cdot 2^{l-1}$$

Step 5: Factor out the invariant term $(k - 1)$ from the geometric series summation:

$$RF_L = 1 + (k - 1) \sum_{l=1}^L 2^{l-1}$$

Step 6: Shift the summation index bounds by setting $j = l - 1$, meaning j goes from 0 to $L - 1$:

$$RF_L = 1 + (k - 1) \sum_{j=0}^{L-1} 2^j$$

Step 7: Apply the standard closed-form algebraic identity for a finite geometric progression sum ($\sum_{j=0}^{n-1} q^j = \frac{1-q^n}{1-q}$):

$$\sum_{j=0}^{L-1} 2^j = \frac{1-2^L}{1-2} = \frac{1-2^L}{-1} = 2^L - 1$$

Step 8: Substitute this closed-form calculation back into our main structural expression:

$$RF_L = 1 + (k - 1) (2^L - 1)$$

Step 9: Expand the algebraic multiplication terms across the brackets:

$$RF_L = 1 + k \cdot 2^L - k - 2^L + 1 = 2^L(k - 1) - k + 2$$

Receptive Field Implications for the CRB Desk

This architectural framework allows the network's receptive field to scale exponentially with depth ($O(2^L)$) while the parameter footprint grows only linearly ($O(L \cdot k)$). This configuration enables a deep TCN to process thousands of high-frequency order book updates (tick-level microstructure) while retaining a memory window broad enough to capture macro-level daily volatility trends.

Proximal Policy Optimization (PPO): The Clipped Surrogate Objective

When training an execution agent to trade large blocks of stock, vanilla Policy Gradient methods often suffer from high variance and destructive parameter shifts. Proximal Policy Optimization (PPO) stabilizes training by regularizing updates using a clipped surrogate objective function:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Line-by-Line Dissection of Objective Terms

- **The Probability Ratio ($r_t(\theta)$):**

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

This term measures the relative probability change of selecting execution action a_t in state s_t under the updated parameter vector θ versus the baseline policy θ_{old} . If $r_t(\theta) > 1$, the action has become more probable under the new policy.

- **The Estimated Advantage Term ($\hat{A}_t$):**

$$\hat{A}_t = G_t - V_\phi(s_t)$$

This represents the baseline-adjusted return, comparing the empirical cumulative reward G_t against the expected value baseline $V_\phi(s_t)$ estimated by the critic network. A positive advantage ($\hat{A}_t > 0$) indicates that the agent's inventory tracking error or execution cost outperformed the historical baseline.

- **The Unclipped Objective ($r_t(\theta) \hat{A}_t$):** This term scales the policy gradient update linearly with the probability ratio, matching standard conservative policy frameworks.
- **The Clipped Objective ($\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t$):** This component bounds the probability ratio within a strict interval around 1:

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \text{if } r_t(\theta) < 1 - \epsilon \\ r_t(\theta) & \text{if } 1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon \\ 1 + \epsilon & \text{if } r_t(\theta) > 1 + \epsilon \end{cases}$$

The hyperparameter ϵ (typically set between 0.1 and 0.2) limits how much the policy can deviate from the old baseline in a single update step.

- **The Outer Element Minimum Operator ($\min(\dots)$):** By taking the minimum of the unclipped and clipped objectives, the loss function acts as a conservative lower bound. It penalizes overly aggressive policy shifts that try to claim unrealistic rewards, while allowing the model to quickly undo poor choices that hurt performance.

Directional Asymmetry Analysis Matrix

Advantage Direction	Ratio Range	Active Function Branch	Gradient Operational Result
$\hat{A}_t > 0$ (Positive Alpha)	$r_t(\theta) > 1 + \epsilon$	Clipped Point: $(1 + \epsilon) \hat{A}_t$	Zero Gradient: Stops updating to prevent over-optimizing.
$\hat{A}_t > 0$ (Positive Alpha)	$r_t(\theta) \leq 1 + \epsilon$	Unclipped Point: $r_t(\theta) \hat{A}_t$	Positive Gradient: Incentivizes selecting this trade path.
$\hat{A}_t < 0$ (Cost Drag)	$r_t(\theta) < 1 - \epsilon$	Unclipped Point: $r_t(\theta) \hat{A}_t$	Negative Gradient: Corrects the policy away from this action.
$\hat{A}_t < 0$ (Cost Drag)	$r_t(\theta) \geq 1 - \epsilon$	Clipped Point: $(1 - \epsilon) \hat{A}_t$	Zero Gradient: Structural floor reached.

Mathematical Formalization of Non-Stationary MDPs

Standard Reinforcement Learning assumes optimization over a stationary **Markov Decision Process (MDP)** defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. Under live market conditions, this stationarity assumption breaks down due to changing macro regimes, liquidity shifts, and behavioral variations among participants.

A **Non-Stationary Markov Decision Process** formalizes this environment by adding time-dependent transition distributions and reward maps:

$$\mathcal{M}_t = (\mathcal{S}, \mathcal{A}, \mathcal{P}_t, \mathcal{R}_t, \gamma)$$

- $\mathcal{S}$ is the operational state space (e.g., current inventory balance, time remaining in trading horizon, bid-ask spread, order book imbalance).
- $\mathcal{A}$ is the action space (e.g., target participation rate, limit order placement offsets).

- $\mathcal{P}_t(s_{t+1}|s_t, a_t)$ is the time-varying transition probability tensor. It defines the probability of moving from state s_t to s_{t+1} after executing action a_t at time t .
- $\mathcal{R}_t(s_t, a_t)$ is the time-varying reward function mapping current execution efficacy to a performance score.

The Non-Stationary Bellman Optimality Equation Breakdown

Under non-stationary dynamics, the optimal action-value function $Q_t^*(s, a)$ must account for future shifts in state transitions and reward distributions:

$$Q_t^*(s, a) = \mathcal{R}_t(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_t(s'|s, a) \max_{a'} Q_{t+1}^*(s', a')$$

Step-by-Step Breakdown of Value Function Degradation

1. Suppose an execution policy π is optimized under an estimated baseline market regime characterized by static matrices $\mathcal{P}_0$ and $\mathcal{R}_0$.
2. The agent optimizes its expected long-term value via policy iterations based on this baseline:

$$V_0^\pi(s) = \mathbb{E}_{\mathcal{P}_0, \mathcal{R}_0} \left[\sum_{\tau=0}^{\infty} \gamma^\tau r_\tau \mid s_0 = s \right]$$

3. Now, suppose a structural macro regime shift occurs at time $t = T$, transitioning the environment to a new state space regime defined by $\mathcal{P}_T \neq \mathcal{P}_0$ and $\mathcal{R}_T \neq \mathcal{R}_0$.
4. The true expected reward achieved by continuing to follow the obsolete baseline policy π under the new regime is governed by:

$$V_T^\pi(s) = \mathbb{E}_{\mathcal{P}_T, \mathcal{R}_T} [r_T + \gamma V_{T+1}^\pi(s') \mid s_T = s]$$

5. The systemic estimation error (or tracking drift) is defined as:

$$\Delta V_T(s) = |V_0^\pi(s) - V_T^\pi(s)|$$

Because the agent updates its value estimates using past data collected under $\mathcal{P}_0$, its internal Q -values diverge from reality. If the market shift increases volatility, this mismatch can cause execution policies to fail, leading to unexpected inventory accumulation and unhedged risk exposure.

3. Production-Grade Python Implementation & Simulation Platform

This production-grade script constructs a full multi-layer **Temporal Convolutional Network (TCN)** architecture with dilated causal processing layers, computes the first-principles **PPO Clipped Surrogate Loss**, and simulates an execution trajectory under non-stationary market regimes. The final evaluation generates an interactive diagnostic panel using Plotly.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# -----
# 1. PRODUCTION TCN SEQUENCE ARCHITECTURE
# -----

class CausalDilatedBlock(nn.Module):
    """
    Implements a single 1D Causal Convolutional Layer with adjustable dilation
    and padding mechanisms to completely prevent future data leakage.
    """
    def __init__(self, in_channels: int, out_channels: int, kernel_size: int, dilation: int):
        super(CausalDilatedBlock, self).__init__()
        self.kernel_size = kernel_size
        self.dilation = dilation

        # Calculate causal padding to ensure output sequence length equals input sequence length
        self.causal_padding = (kernel_size - 1) * dilation

        self.conv = nn.Conv1d(
            in_channels=in_channels,
            out_channels=out_channels,
            kernel_size=kernel_size,
            padding=self.causal_padding,
            dilation=dilation
        )
```

```

self.activation = nn.Mish()
self.norm = nn.BatchNorm1d(out_channels)
self.dropout = nn.Dropout(p=0.15)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Input shape: [Batch, Channels, Sequence Length]
    Output shape: [Batch, Channels, Sequence Length]
    """
    res = x
    out = self.conv(x)
    # Chop off the trailing edge padding entries to enforce temporal causality
    if self.causal_padding > 0:
        out = out[:, :, :-self.causal_padding]
    out = self.norm(out)
    out = self.activation(out)
    out = self.dropout(out)

    # Apply a projection shortcut layer if dimensions change
    if res.shape[1] != out.shape[1]:
        shortcut_layer = nn.Conv1d(res.shape[1], out.shape[1], kernel_size=1).to(x.device)
        res = shortcut_layer(res)

    return out + res

class CRBTemporalNetwork(nn.Module):
    """
    Temporal Convolutional Network (TCN) optimized for high-frequency order book tracking.
    Uses an exponential dilation schedule to expand the receptive field efficiently.
    """
    def __init__(self, input_features: int, hidden_dim: int, num_layers: int = 4):
        super(CRBTemporalNetwork, self).__init__()
        self.layers = nn.ModuleList()

        # Layer 1 block configuration
        self.layers.append(CausalDilatedBlock(input_features, hidden_dim, kernel_size=3, dilation=1))

        # Construct an exponential dilation hierarchy: d = 2, 4, 8, 16...
        for l in range(1, num_layers):
            self.layers.append(
                CausalDilatedBlock(hidden_dim, hidden_dim, kernel_size=3, dilation=2**l)
            )

        # Actor and Critic heads
        self.actor_head = nn.Linear(hidden_dim, 3) # Continuous actions mapped to 3 discrete intervals
        self.critic_head = nn.Linear(hidden_dim, 1) # State-value baseline tracker

    def forward(self, x: torch.Tensor) -> tuple:
        """
        Forward pass processing for sequence inputs.
        Shape expectation: [Batch, Sequence Length, Features]
        """
        # Permute tensor formatting to match PyTorch Conv1D channel expectations
        x = x.permute(0, 2, 1)

        for layer in self.layers:
            x = layer(x)

        # Extract the most recent time-step element vector to guide real-time deployment
        latest_state = x[:, :, -1]

        policy_logits = self.actor_head(latest_state)
        state_values = self.critic_head(latest_state)

        return policy_logits, state_values

# -----
# 2. PPO FIRST-PRINCIPLES CRITICAL OPTIMIZER ENGINE
# -----

class PPOClippedLossEvaluator:
    """

```

Computes the PPO clipped surrogate objective from first principles, incorporating value function clipping and tracking metrics.

"""

```
def __init__(self, clip_epsilon: float = 0.2, entropy_coefficient: float = 0.01):
    self.eps = clip_epsilon
    self.entropy_coef = entropy_coefficient
```

```
def evaluate_loss(self,
                 old_log_probs: torch.Tensor,
                 current_logits: torch.Tensor,
                 actions: torch.Tensor,
                 advantages: torch.Tensor,
                 old_values: torch.Tensor,
                 current_values: torch.Tensor,
                 returns: torch.Tensor) -> tuple:
    """
    Computes the complete, self-contained mathematical loss optimization profile.
    """
    # Determine current probability distributions via Softmax
    probs = torch.softmax(current_logits, dim=-1)
    log_probs = torch.log_softmax(current_logits, dim=-1)

    # Isolate the log probabilities of the selected execution actions
    gathered_log_probs = log_probs.gather(1, actions.unsqueeze(1)).squeeze(1)

    # Calculate the probability ratio r_t(theta) from first principles
    ratio = torch.exp(gathered_log_probs - old_log_probs)

    # Compute the unclipped and clipped surrogate objective components
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1.0 - self.eps, 1.0 + self.eps) * advantages

    # The primary policy gradient loss component is the negative expected minimum
    policy_loss = -torch.mean(torch.min(surr1, surr2))

    # Value Function Clipping (Critic Loss Component)
    value_pred_clipped = old_values + torch.clamp(current_values - old_values, -self.eps, self.eps)
    vf_loss1 = (current_values - returns) ** 2
    vf_loss2 = (value_pred_clipped - returns) ** 2
    critic_loss = 0.5 * torch.mean(torch.max(vf_loss1, vf_loss2))

    # Shannon Entropy calculation to incentivize environmental exploration
    entropy = -torch.sum(probs * log_probs, dim=-1)
    entropy_loss = -torch.mean(entropy)

    # Total joint multi-task objective combination
    total_loss = policy_loss + 1.0 * critic_loss + self.entropy_coef * entropy_loss

    return total_loss, policy_loss, critic_loss, ratio.detach().cpu().numpy()
```

```
# -----
# 3. NON-STATIONARY CRB ENVIRONMENT SIMULATION
# -----
```

```
def generate_crb_trajectory(steps: int = 400) -> pd.DataFrame:
    """
    Simulates high-frequency order-book data with an abrupt structural
    regime shift to evaluate model robustness under non-stationary conditions.
    """
    np.random.seed(13458) # Nomura Job Code identifier seed

    # State tracking dimensions
    imbalance = np.random.normal(0, 1, steps)
    spread = np.random.uniform(0.01, 0.05, steps)
    inventory = np.zeros(steps)
    rewards = np.zeros(steps)
    advantages = np.zeros(steps)
    ratios_simulated = np.zeros(steps)

    current_inv = 10000.0 # Starting asset volume inventory block

    # Define a transition point where market conditions shift
    regime_shift_idx = 200
```

```

for t in range(steps):
    if t < regime_shift_idx:
        # Regime 1: Mean-reverting, highly liquid framework
        execution_cost_factor = 0.02
        adverse_selection = 0.05 * imbalance[t]
    else:
        # Regime 2: Distorted, high-volatility liquidity crunch
        execution_cost_factor = 0.08
        adverse_selection = 0.35 * imbalance[t]
        spread[t] += 0.04

    # Simulated continuous execution choices
    action_rate = 0.05 + 0.02 * np.sin(t / 10.0)
    executed_shares = current_inv * action_rate
    current_inv -= executed_shares
    inventory[t] = current_inv

    # Compute multi-component transaction cost execution rewards
    cost_drag = (executed_shares * spread[t] * execution_cost_factor) + abs(executed_shares * adverse_selection)
    rewards[t] = -cost_drag

    # Calculate tracking variations and advantage metrics
    advantages[t] = rewards[t] - (-150.0 if t < regime_shift_idx else -650.0)
    ratios_simulated[t] = 1.0 + np.random.normal(0, 0.05 if t < regime_shift_idx else 0.28)

return pd.DataFrame({
    "Step": np.arange(steps),
    "Imbalance": imbalance,
    "Spread": spread,
    "Inventory": inventory,
    "Reward": rewards,
    "Advantage": advantages,
    "SimulatedRatio": ratios_simulated,
    "Regime": ["Stationary Base" if x < regime_shift_idx else "Volatility Shock" for x in range(steps)]
})

# -----
# 4. COMPREHENSIVE ENGINE EXECUTION AND VISUALIZATION
# -----

if __name__ == "__main__":
    print("="*70)
    print("      INITIALIZING NOMURA CRB DEEP RL DIAGNOSTICS PLATFORM      ")
    print("="*70)

    # Instantiate structural networks
    model_engine = CRBTemporalNetwork(input_features=3, hidden_dim=32, num_layers=4)
    loss_calculator = PPOClippedLossEvaluator(clip_epsilon=0.2)

    # Generate simulated non-stationary trajectory data
    trajectory_data = generate_crb_trajectory(steps=400)

    # Format and pass a slice of data through the TCN network
    mock_input_batch = torch.randn(10, 50, 3) # Batch size 10, Sequence Length 50, Features 3
    logits, values = model_engine(mock_input_batch)

    print(f"Executed TCN Pipeline Architecture Successfully.")
    print(f"Actor Tensor Dimensions : {list(logits.shape)} (Logits mapping to choices)")
    print(f"Critic Tensor Dimensions : {list(values.shape)} (Value function expectation state baseline)")
    print("-"*70)

    # Process PPO first-principles evaluation metrics over simulated trajectories
    old_log_p = torch.tensor(np.log(1.0 / 3.0) + np.random.normal(0, 0.1, 100), dtype=torch.float32)
    curr_log_l = torch.randn(100, 3, dtype=torch.float32)
    actions_vector = torch.randint(0, 3, (100,))
    adv_vector = torch.tensor(trajectory_data["Advantage"].values[:100], dtype=torch.float32)
    old_v = torch.randn(100, dtype=torch.float32)
    curr_v = torch.randn(100, dtype=torch.float32)
    ret_vector = curr_v + adv_vector

    t_loss, p_loss, c_loss, empirical_ratios = loss_calculator.evaluate_loss(
        old_log_p, curr_log_l, actions_vector, adv_vector, old_v, curr_v, ret_vector
    )

```

```

)

print("PPO FIRST-PRINCIPLES GRADIENT BREAKDOWN:")
print(f"Calculated Total Optimization Objective Loss Score : {t_loss.item():.4f}")
print(f"Evaluated Policy Gradient Component Loss          : {p_loss.item():.4f}")
print(f"Evaluated Critic State Value Tracking Loss         : {c_loss.item():.4f}")
print("=*70)

# -----
# 5. DYNAMIC PLOTLY DIAGNOSTIC PANEL COMPILATION
# -----

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=(
        "Inventory Decay Curve & Execution Track",
        "Transaction Cost Reward Degradation Profile",
        "Empirical Advantage Drift Under Non-Stationary Shifts",
        "PPO Policy Ratio Deviation Distribution Space"
    ),
    vertical_spacing=0.15, horizontal_spacing=0.10
)

# Panel 1: Inventory Decay
fig.add_trace(
    go.Scatter(
        x=trajectory_data["Step"], y=trajectory_data["Inventory"],
        mode="lines", name="Block Inventory Volume",
        line=dict(color="#002147", width=2.5)
    ), row=1, col=1
)
# Vertical line highlighting the regime shift point
fig.add_shape(
    type="line", x0=200, x1=200, y0=0, y1=10000,
    line=dict(color="red", width=2, dash="dash"), row=1, col=1
)

# Panel 2: Transaction Cost Reward
fig.add_trace(
    go.Scatter(
        x=trajectory_data["Step"], y=trajectory_data["Reward"],
        mode="lines", name="Instantaneous P&L Drag",
        line=dict(color="#D96B27", width=2)
    ), row=1, col=2
)
fig.add_shape(
    type="line", x0=200, x1=200, y0=trajectory_data["Reward"].min(), y1=0,
    line=dict(color="red", width=2, dash="dash"), row=1, col=2
)

# Panel 3: Advantage Drift
fig.add_trace(
    go.Scatter(
        x=trajectory_data["Step"], y=trajectory_data["Advantage"],
        mode="lines", name="Advantage Metric Delta",
        line=dict(color="#2CA02C", width=1.5)
    ), row=2, col=1
)
fig.add_shape(
    type="line", x0=0, x1=400, y0=0, y1=0,
    line=dict(color="black", width=1, dash="dot"), row=2, col=1
)
fig.add_shape(
    type="line", x0=200, x1=200, y0=trajectory_data["Advantage"].min(), y1=trajectory_data["Advantage"].max(),
    line=dict(color="red", width=2, dash="dash"), row=2, col=1
)

# Panel 4: PPO Policy Ratio Deviation Bounds
fig.add_trace(
    go.Scatter(
        x=trajectory_data["Step"], y=trajectory_data["SimulatedRatio"],
        mode="markers", name="r_t(theta) Observations",
        marker=dict(color="#4682B4", size=4, opacity=0.6)
    ), row=2, col=2
)

```

```

)
# Upper and Lower PPO Clipping Boundary indicators
fig.add_shape(
    type="line", x0=0, x1=400, y0=1.2, y1=1.2,
    line=dict(color="purple", width=1.5, dash="dot"), row=2, col=2
)
fig.add_shape(
    type="line", x0=0, x1=400, y0=0.8, y1=0.8,
    line=dict(color="purple", width=1.5, dash="dot"), row=2, col=2
)
fig.add_shape(
    type="line", x0=200, x1=200, y0=0.4, y1=1.8,
    line=dict(color="red", width=2, dash="dash"), row=2, col=2
)

# Set global layout properties
fig.update_layout(
    title=dict(
        text="<b>Nomura Quantitative Research Sequence Platform Panel</b><br><sup>Evaluating TCN Execution & PPO  
Bound Mechanics Under Liquidity Non-Stationarity</sup>",
        x=0.05, font=dict(size=16, family="Arial")
    ),
    template="plotly_white", height=760, width=1150, showlegend=False
)

fig.update_xaxes(title_text="Intraday Time Steps (Ticks)", row=1, col=1)
fig.update_xaxes(title_text="Intraday Time Steps (Ticks)", row=1, col=2)
fig.update_xaxes(title_text="Intraday Time Steps (Ticks)", row=2, col=1)
fig.update_xaxes(title_text="Intraday Time Steps (Ticks)", row=2, col=2)

fig.update_yaxes(title_text="Remaining Shares", row=1, col=1)
fig.update_yaxes(title_text="Execution Penalty Reward ($)", row=1, col=2)
fig.update_yaxes(title_text="Advantage Estimate Score", row=2, col=1)
fig.update_yaxes(title_text="Probability Ratio  $r_t(\theta)$ ", row=2, col=2)

# To view the interactive Plotly dashboard, uncomment the line below:
# fig.show()

```